

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: V.Lekhya sri

Roll No.: A24126552123

Date:

1. TITLE

Develop a Student Profile Generator Using JavaScript Classes and DOM Manipulation

2. AIM

To develop a mini web application using **JavaScript** that accepts student information through an HTML form, creates a **Student object using a JavaScript class**, and dynamically displays the submitted details using **DOM manipulation**.

3. OBJECTIVE

The objectives of this experiment are:

- To understand JavaScript classes and objects.
 - To create a Student class with properties such as Name, Roll Number, Department, and CGPA.
 - To create a Student object using user-provided values.
 - To understand DOM selection and manipulation.
 - To dynamically create and display HTML elements.
 - To implement event handling using a button click.
 - To apply CSS styling to the dynamically generated student profile.
-

4. THEORY

JavaScript is a programming language used to add dynamic and interactive functionality to web pages.

JavaScript Classes provide a blueprint for creating objects with properties and methods. In this application, a **Student class** is used to store student information such as name, roll number, department, and CGPA.

DOM (Document Object Model) represents the HTML document as a tree of objects, allowing JavaScript to access and modify webpage elements dynamically.

In this application:

- `class Student` is used to define the structure of a student object.
- `constructor()` initializes the student properties.
- `new Student()` creates a Student object using user-provided values.
- `document.getElementById()` is used to select HTML elements from the DOM.
- `.value` retrieves the values entered by the user.
- `addEventListener()` handles the button click event.
- `document.createElement()` dynamically creates new HTML elements.
- `textContent` adds student information to the dynamically created elements.
- `appendChild()` adds the created elements to the webpage.

The basic flow of the application is:

User Input → Button Click → Student Object → DOM Manipulation → Student Profile Display

5. ALGORITHM / PROCEDURE

1. Create an HTML file containing input fields for Name, Roll Number, Department, and CGPA.
 2. Add a **Create Profile** button to the webpage.
 3. Create a JavaScript file and define a Student class.
 4. Define the `constructor()` to initialize Name, Roll Number, Department, and CGPA.
 5. Select the Create Profile button using `document.getElementById()`.
 6. Add a click event listener to the button.
 7. Retrieve the values entered by the user from the input fields.
 8. Create a Student object using the new `Student()` constructor.
 9. Select the profile container from the DOM using `document.getElementById()`.
 10. Retrieve the profile HTML structure from the Student object using the `getProfile()` method.
 11. Display the generated student profile by setting the container's `innerHTML` property.
 12. Verify the output by entering different student details.
 13. Compare the actual output with the expected output.
-

6. PROGRAM

script.js

```
class Student {
    constructor(name, rollNumber, department, cgpa) {
        this.name = name;
        this.rollNumber = rollNumber;
        this.department = department;
        this.cgpa = cgpa;
    }
}
```

```

    }

    getProfile() {
        return `
            <div class="student-card">
                <h2>Student Profile</h2>

                <div class="detail">
                    <span>Name</span>
                    <span>: ${this.name}</span>
                </div>

                <div class="detail">
                    <span>Roll No</span>
                    <span>: ${this.rollNumber}</span>
                </div>

                <div class="detail">
                    <span>Department</span>
                    <span>: ${this.department}</span>
                </div>

                <div class="detail">
                    <span>CGPA</span>
                    <span>: ${this.cgpa}</span>
                </div>
            </div>
        `;
    }
}

function generateProfile() {

    const name = document.getElementById("studentName").value;
    const rollNumber = document.getElementById("studentRoll").value;
    const department = document.getElementById("studentDept").value;
    const cgpa = document.getElementById("studentCgpa").value;

    if (name === "" || rollNumber === "" ||
        department === "" || cgpa === "") {

        alert("Please enter all student details.");
        return;
    }
}

```

```

    }

    const student = new Student(
        name,
        rollNumber,
        department,
        cgpa
    );

    const output = document.getElementById("output");

    output.innerHTML = student.getProfile();
}

```

studentdetails.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Student Profile</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <div class="wrapper">
        <h1>Student Profile</h1>

        <div class="input-box">
            <input type="text" id="studentName" placeholder="Name">
            <input type="text" id="studentRoll" placeholder="Roll
Number">
            <input type="text" id="studentDept"
placeholder="Department">
            <input type="number" id="studentCgpa" placeholder="CGPA"
step="0.01">

```

```
        <button onclick="generateProfile()">Display  
Profile</button>  
    </div>  
  
    <div id="output"></div>  
</div>  
  
    <script src="script.js"></script>  
  
</body>  
</html>
```

style.css

```
body {  
    margin: 0;  
    font-family: Arial, sans-serif;  
    background: #eeeeee;  
}  
  
.wrapper {  
    width: 550px;  
    margin: 50px auto;  
    text-align: center;  
}  
  
h1 {  
    margin-bottom: 25px;  
}  
  
.input-box {  
    background: white;  
    padding: 25px;  
    border-radius: 12px;  
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);  
}  
  
input {  
    display: block;  
    width: 90%;  
    padding: 12px;
```

```
    margin: 10px auto;
    border: 1px solid #aaa;
    border-radius: 6px;
    font-size: 15px;
}

button {
    padding: 12px 25px;
    margin-top: 10px;
    border: none;
    border-radius: 6px;
    background: #222;
    color: white;
    font-size: 15px;
    cursor: pointer;
}

button:hover {
    background: #444;
}

.student-card {
    margin-top: 30px;
    padding: 25px;
    background: white;
    border-radius: 12px;
    text-align: left;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.15);
}

.student-card h2 {
    text-align: center;
    margin-bottom: 25px;
}

.detail {
    display: grid;
    grid-template-columns: 120px 1fr;
    padding: 10px 0;
    font-size: 17px;
    border-bottom: 1px solid #ddd;
}
```

```
.detail span:first-child {  
    font-weight: bold;  
}
```

7. CODE EXPLANATION

Code	Explanation
<code>class Student</code>	Defines a JavaScript class named <code>Student</code> to represent student information.
<code>constructor(name, rollNumber, department, cgpa)</code>	Initializes the properties of a <code>Student</code> object.
<code>this.name = name</code>	Stores the student's name in the object.
<code>this.rollNumber = rollNumber</code>	Stores the student's roll number.
<code>this.department = department</code>	Stores the student's department.
<code>this.cgpa = cgpa</code>	Stores the student's CGPA.
<code>getProfile()</code>	Returns a template string containing the formatted HTML markup for the student profile card using the instance properties.
<code>generateProfile()</code>	Handles form submission by reading user input values, validating them, creating a <code>Student</code> instance, and updating the DOM.
<code>document.getElementById("studentName").value</code>	Retrieves the name entered by the user.
<code>document.getElementById("studentRoll").value</code>	Retrieves the roll number entered by the user.
<code>document.getElementById("studentDept").value</code>	Retrieves the department entered by the user.
<code>document.getElementById("studentCgpa").value</code>	Retrieves the CGPA entered by the user.
<code>new Student(name, rollNumber, department, cgpa)</code>	Creates a new <code>Student</code> object using the user-provided values.

<pre>output.innerHTML = student.getProfile()</pre>	Sets the inner HTML of the target output container directly to the template string generated by the Student object's <code>getProfile()</code> method.
--	--

8. TEST CASES AND EXECUTION DATA

Test Case	Test Description	Expected Result	Status
TC01	Enter all valid student details and click Display Profile	Student profile is displayed with all entered details	Pass
TC02	Leave one or more fields empty	Alert message asks to enter all student details	Pass
TC03	Enter a decimal CGPA value	CGPA is accepted and displayed correctly	Pass
TC04	Click Display Profile multiple times with valid data	Updated student profile is displayed correctly	Pass

9.OUTPUT

Student Profile

Display Profile

Student Profile

Name	: V lekhy sri
Roll No	: 123
Department	: csm
CGPA	: 85

Student Profile

Display Profile

Please enter all student details.

Close

Student Profile

lekhyaa

123

csm

8.25

Display Profile

Student Profile

Name : lekhyaa

Roll No : 123

Department : csm

CGPA : 8.25

Student Profile

Shavan

85

csm

8.25

Display Profile

Student Profile

Name : Shavan

Roll No : 85

Department : csm

CGPA : 8.25

Student Profile

Gouri

76

csm

8.25

Display Profile

Student Profile

Name : Gouri

Roll No : 76

Department : csm

CGPA : 8.25

10. RESULT

Thus, a **Student Profile Generator** was successfully developed using **JavaScript classes and DOM manipulation**. A Student class was created, user-provided student details were used to create objects, and the profile was dynamically generated and displayed on the webpage. The application was successfully verified using multiple test cases.
